



Klickt Gern uf Sache

PostFinance – Beyond the List

Technische Informationen für die Jury



Technische Informationen für die Jury

Aktueller Stand des Sourcecodes

- <https://github.com/pandermatt/bern-hackt-2026>
- **App:** <https://beyond-money.ch/> & <https://dev.beyond-money.ch/>

Email

jeanine@example.com

Password

beyond-money-demo



Ausgangslage

- Fokus: Aus Kontoauszügen konkrete, verständliche Finanz-Insights machen. Beyond Money zeigt Einnahmen, Ausgaben, Budgets, Sparziele, Anomalien und beantwortet Fragen zu den eigenen Zahlen.
- Grundsatzentscheid: Progressive Web App PWA mit Next.js als Full-Stack-Monolith statt separatem Frontend und Backend. Seiten, Server Actions, Authentifizierung und Geschäftslogik liegen in einer gemeinsamen TypeScript-Codebasis.
- Grundsatzentscheid: deterministische Berechnungen vor generativer KI. Beträge, Aggregationen, Budgets und Auffälligkeiten entstehen aus TypeScript-/SQL-Code; *Apertus erklärt, verdichtet und fragt kontrollierte Werkzeuge ab.*
- Grundsatzentscheid: Datenschutz durch serverseitige Mandantentrennung. Die angemeldete Person wird aus der Session bestimmt; Datenbankabfragen sind konsequent auf dieses Konto begrenzt.

Technischer Aufbau

- **Frontend und App:** Next.js 16, React 19, TypeScript, Tailwind v4 und shadcn/ui. Server Components laden Daten direkt; Client Components werden nur für interaktive Teile wie Diagramme, Filter und Chat eingesetzt. Manifest und Service Worker machen die App installierbar (=PWA).
- **Push Notifications:** Push Notifications werden gesendet, sobald die App eine Auffälligkeit erkennt hat. Für dies muss die PWA installiert sein.
- **Datenhaltung:** SQLite mit better-sqlite3 angebunden über Drizzle ORM. Die Datenbank enthält Benutzer, gehashte Sessions, normalisierte Transaktionen, Budgets, Sparziele, monatliche Zuweisungen und persistierte Anomalien.
- **Visualisierung:** Apache ECharts 6 rendert die Diagramme. Die Daten werden serverseitig aggregiert; dieselben Aggregationen werden auch vom Assistenten verwendet.
- **I18 Internationalisierung und UI:** next-intl mit Deutsch und Englisch, Hell-/Dunkelmodus und Screenreader-Tabellen zu den Diagrammen.
- **AI-Schicht:** Apertus v1.5 8B/70B läuft serverseitig. Das Modell erhält keine Rohdaten als Prompt, sondern ruft kontrollierte Werkzeuge auf und arbeitet mit geprüften Aggregaten.
- **Deployment und Qualität:** GitHub Actions prüfen Linting, Production-Build/Typecheck und die Testsuite. Deployment über Coolify auf einem Hetzner-Server; main läuft auf beyond-money.ch, development auf dev.beyond-money.ch.
- **Nutzung von AI:** Zur Verbesserung des Codes und bei der Codeentwicklung (z. B. durch Autovervollständigung, Boilerplate-Code und Hilfsskripte wie Datenbank-Setup-Tools) haben wir generative AI-Tools (darunter ChatGPT, Claude, GitHub Copilot und Gemini) eingesetzt. Alle Ergebnisse wurden von den Autoren sorgfältig geprüft, validiert und überarbeitet.

Implementation

- **Import:** CSV-Zeilen werden normalisiert, über natürliche Schlüssel dedupliziert und Händlern sowie Kategorien zugeordnet. Dadurch werden doppelte Kreditkartenbuchungen erkannt und Schreibvarianten wie „Orell Fuessli“ und „Orell Füssli“ zusammengeführt.

- **Anomalie-Algorithmus:** 26 deterministische Regeln erkennen Auffälligkeiten wie doppelte Belastungen, ausbleibende Lohneingänge, wachsende Abos oder ungewöhnliche Transfers. Ein Hintergrundscan persistiert Resultate und Fortschritt.
- **AI mit Leitplanken:** Apertus kann Befunde verständlich zusammenfassen und im Chat Werkzeuge aufrufen, ist aber nie Quelle der Finanzzahlen. Bei Timeout, ungültigem JSON oder fehlendem Schlüssel fällt die Lösung auf deterministische Regeln zurück.
- **Sicherheit:** Passwörter werden mit script und individuellem Salt gehasht. Session-Cookies enthalten nur Zufallstokens; in der Datenbank liegt deren SHA-256-Hash.

Abgrenzung / Offene Punkte

- **Keine direkte Bankanbindung:** Buchungen werden aus CSV-Dateien importiert. Open Banking wurde zugunsten eines funktionalen Prototyps bewusst nicht umgesetzt; die Importschicht ist dafür vorbereitet.
- **SQLite statt Datenbankserver:** Für den Prototyp genügt eine persistente Datei. Ein Wechsel auf PostgreSQL wäre möglich, braucht aber einen Migrationsplan.
- **Kein eigens trainiertes LLM:** Die Anomalie-Erkennung ist bewusst deterministisch. Ein eigenes Modell hätte den Rahmen des Hackathons gesprengt und wäre für verlässliche Finanzwarnungen weniger transparent.
- **Keine automatische Geldverschiebung:** Sparziel-Vorschläge verändern keine Daten ohne Bestätigung. Der Benutzer löst jede Mutation selbst aus; serverseitig wird sie erneut validiert.

